机器学习

课程目标

- 学习和掌握机器学习基本概念与技术；
- 掌握常用机器学习经典算法；
- 能够初步选用和设计针对特定应用的机器学习解决方案。

课程内容

- 人工智能与机器学习概况
- Python基础
- K近邻
- 模型选择
- 线性回归
- 逻辑回归
- 分类评价指标
- 朴素贝叶斯
- 支持向量机
- 决策树
- 集成学习
- 聚类
- 深度学习

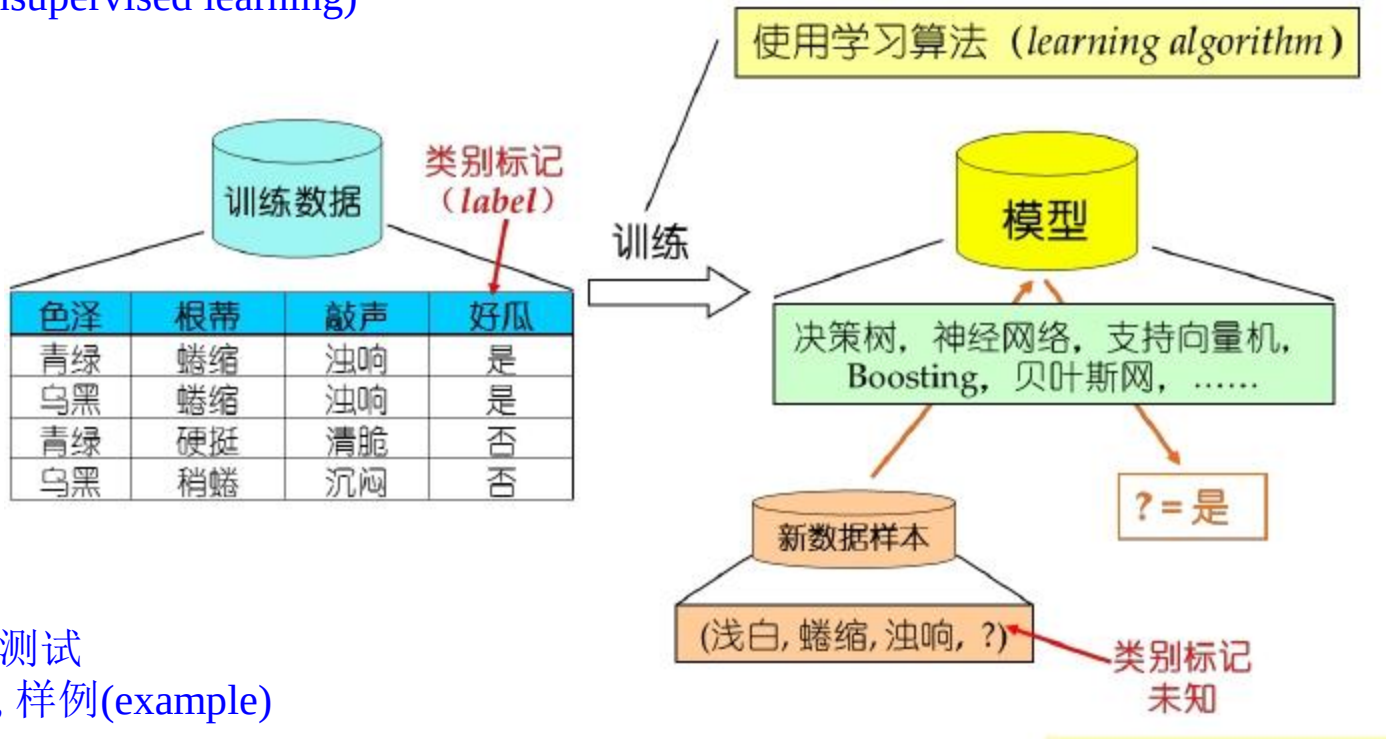
第1章 人工智能简介

- 人工智能概念
- 人工智能简史

第2章 机器学习简介

基本术语

- 监督学习(supervised learning)
- 无监督学习(unsupervised learning)



色泽	根蒂	敲声	好瓜
青绿	蜷缩	浊响	是
乌黑	蜷缩	浊响	是
青绿	硬挺	清脆	否
乌黑	稍蜷	沉闷	否

- 假设(hypothesis)
- 真相(ground-truth)

- 分类, 回归
- 二分类, 多分类
- 正类, 反类

- 数据集; 训练, 测试
- 示例(instance), 样例(example)
- 样本(sample)
- 属性(attribute), 特征(feature); 属性值
- 特征向量(feature vector)

- 未见样本(unseen instance)
- 未知“分布”
- 独立同分布(i.i.d.)
- 泛化**(generalization)**

第3章 Python简介

Python编程基础

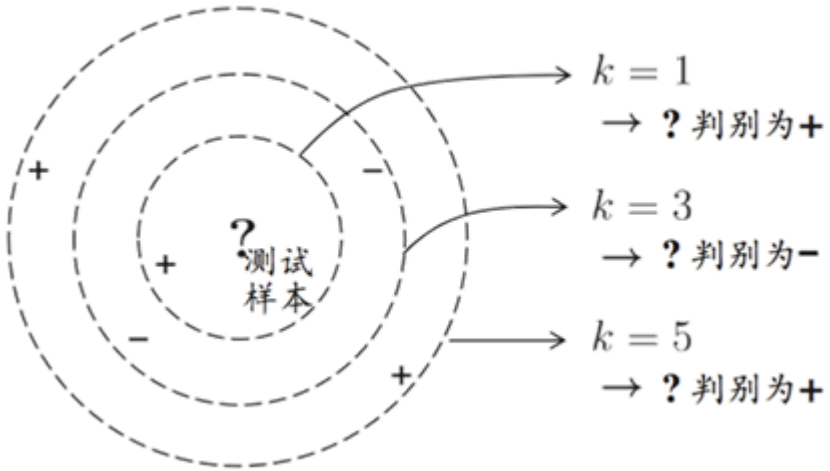
- 数据类型和变量
- 运算符和表达式
- 容器 list set tuple dict 访问，切片
- 控制流语句
- 函数
- 文件输入输出
- 相关库 numpy matplotlib pandas sklearn

第4章 K近邻

k近邻

k近邻(k-Nearest Neighbor, kNN)学习是一种监督学习方法:

- 确定训练样本，以及某种**距离度量**。
- 对于某个给定的测试样本，找到训练集中距离最近的**k**个样本，对于分类问题使用“投票法”获得预测结果，对于回归问题使用“平均法”获得预测结果。还可基于距离远近进行加权平均或加权投票，距离越近的样本权重越大。
  - 投票法：选择这k个样本中出现最多的类别标记作为预测结果。
  - 平均法：将这k个样本的实值输出标记的平均值作为预测结果。



K近邻

导入包含分类方法的类：

```
from sklearn.neighbors import KNeighborsClassifier
```

创建该类的一个对象：

```
KNN = KNeighborsClassifier(n_neighbors=3)
```

拟合数据集，即训练KNN模型，并用训练好的模型预测数据的标签：

```
KNN = KNN.fit(X_data, y_data)
y_predict = KNN.predict(X_data)
```

	sepal-length	sepal-width	petal-length	petal-width	Class
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

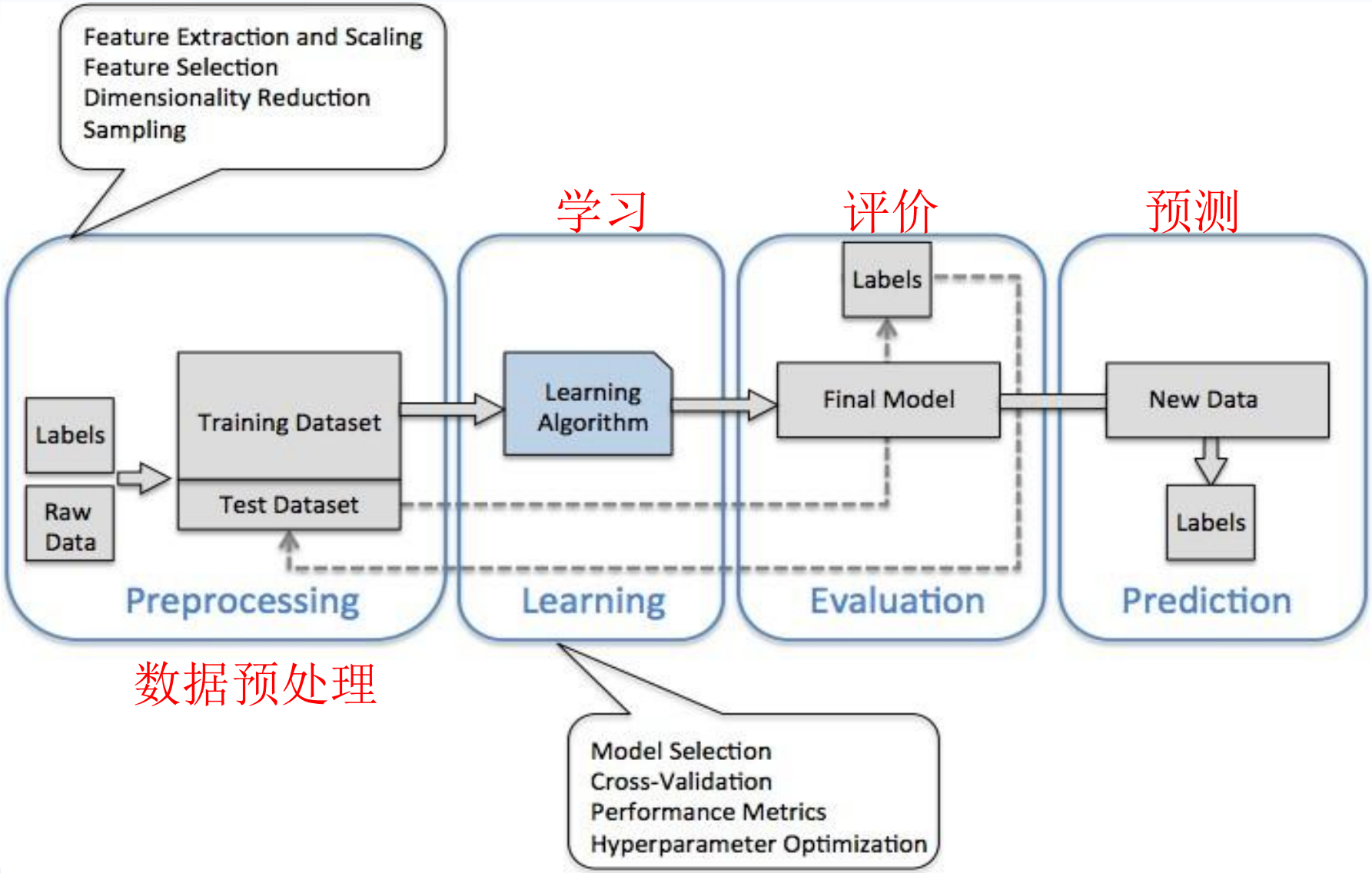
```
X = dataset.iloc[:, :-1].values
y = dataset.iloc[:, 4].values
```

```python
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
scaler.fit(X_train)

X_train = scaler.transform(X_train)
X_test = scaler.transform(X_test)
```

```python
from sklearn.neighbors import KNeighborsClassifier
classifier = KNeighborsClassifier(n_neighbors=5)
classifier.fit(X_train, y_train)
```

创建机器学习应用路线图



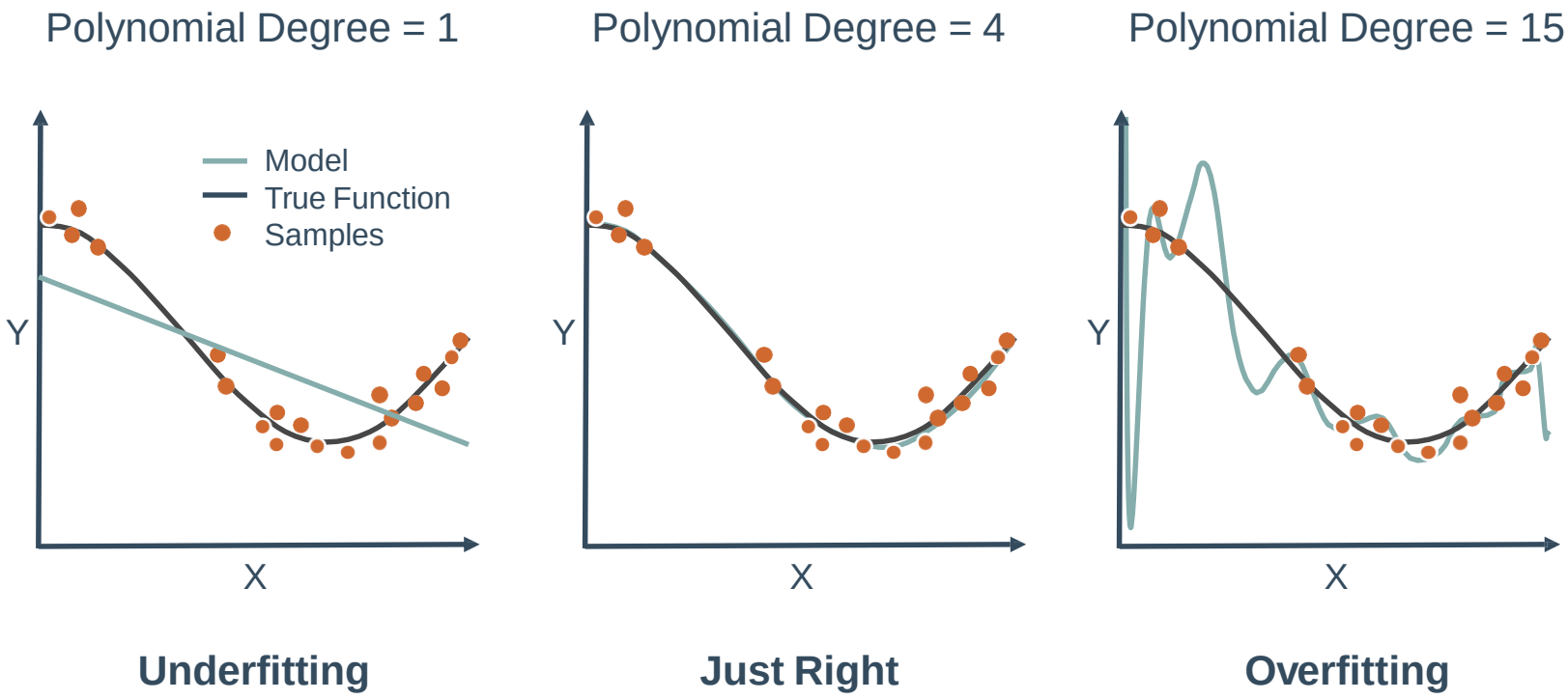
第5章 模型选择

模型评价

训练精度（training accuracy）

- 在整个数据集上训练模型
- 并在同一个数据集上测试模型，得到模型的预测结果，和真实结果做比较，计算模型的精度

欠拟合与过拟合



欠拟合和过拟合都会导致较大的泛化误差。

模型评价

测试精度（testing accuracy）

- 把数据集划分成两个子集：训练集和测试集
- 在训练集上训练模型
- 在测试集上测试模型，并计算精度

- 留出法 (hold-out)
- 交叉验证法 (cross validation)
- 自助法 (bootstrap)

留出法

	Date	Title	Budget	DomesticTotalGross	Director	Rating	Runtime
0	2013-11-22	The Hunger Games: Catching Fire	130000000	424668047	Francis Lawrence	PG-13	146
1	2013-05-03	Iron Man 3	200000000	409013994	Shane Black	PG-13	129
2	2013-11-22	Frozen	150000000	400738009	Chris BuckJennifer Lee	PG	108
3	2013-07-03	Despicable Me 2	76000000	368061265	Pierre CoffinChris Renaud	PG	98
4	2013-06-14	Man of Steel	225000000	291045518	Zack Snyder	PG-13	143
5	2013-10-04	Gravity	100000000	274092705	Alfonso Cuaron	PG-13	91
6	2013-06-21	Monsters University	NaN	268492764	Dan Scanlon	G	107
7	2013-12-13	The Hobbit: The Desolation of Smaug	NaN	258366855	Peter Jackson	PG-13	161
8	2013-05-24	Fast & Furious 6	160000000	238679850	Justin Lin	PG-13	130
9	2013-03-08	Oz The Great and Powerful	215000000	234911825	Sam Raimi	PG	127
10	2013-05-16	Star Trek Into Darkness	190000000	228778661	J.J. Abrams	PG-13	123
11	2013-11-08	Thor: The Dark World	170000000	206362140	Alan Taylor	PG-13	120
12	2013-06-21	World War Z	190000000	202359711	Marc Forster	PG-13	116
13	2013-03-22	The Croods	135000000	187168425	Kirk De MiccoChris Sanders	PG	98
14	2013-06-28	The Heat	43000000	159582188	Paul Feig	R	117
15	2013-08-07	We're the Millers	37000000	150394119	Rawson Marshall Thurber	R	110
16	2013-12-13	American Hustle	40000000	150117807	David O. Russell	R	138
17	2013-05-10	The Great Gatsby	105000000	144840419	Baz Luhrmann	PG-13	143

训练数据

验证数据

超越单个测试集：交叉验证

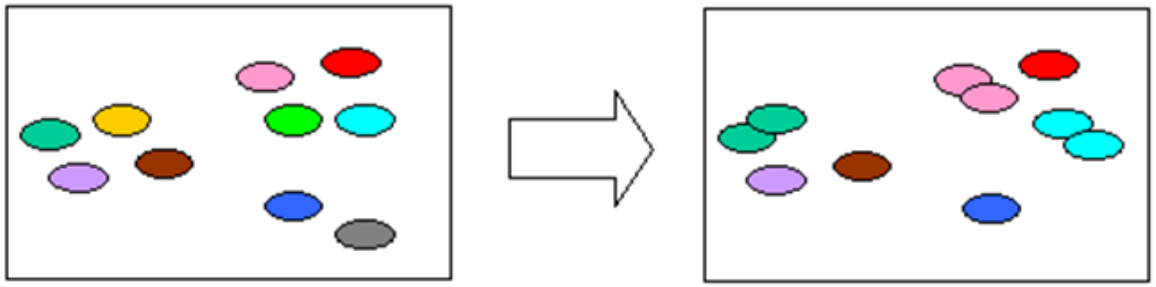
自助法：

以自助采样法为基础，对数据集 D 有放回采样 m 次得到训练集 D'，$D \setminus D'$ 用做测试集。

$$\lim_{m \mapsto \infty} \left(1 - \frac{1}{m}\right)^m \mapsto \frac{1}{e} \approx 0.368$$

- 实际模型与预期模型都使用 m 个训练样本
- 约有1/3的样本没在训练集中出现

- 从初始数据集中产生多个不同的训练集，对集成学习有很大的好处
- 自助法在数据集较小、难以有效划分训练/测试集时很有用；
- 由于改变了数据集分布可能引入估计偏差，在数据量足够时，留出法和交叉验证法更常用。



交叉验证的语法

导入划分训练集和测试集的函数：

```
from sklearn.model_selection import cross_val_score
```

用一个给定的模型执行交叉验证：

```
cross_val = cross_val_score(KNN, X_data, y_data, cv=4,
                            scoring='neg_mean_squared_error')
```

其他CV splitter:

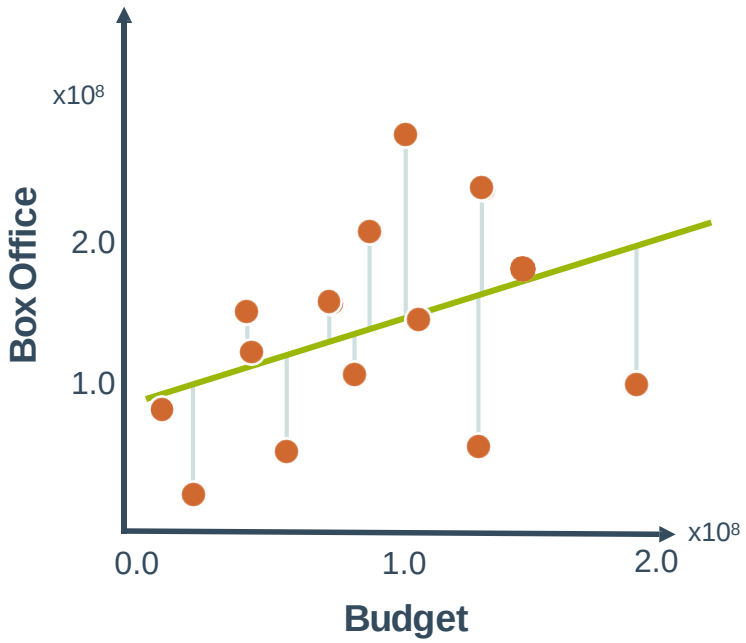
LeaveOneOut, ShuffleSplit, StratifiedShuffleSplit ……

```python
from sklearn.neighbors import KNeighborsClassifier
from sklearn.model_selection import cross_val_score
knn = KNeighborsClassifier(n_neighbors=5)
scores = cross_val_score(knn, X, y, cv=10, scoring='accuracy')
```

```python
k_range = list(range(1, 31))
k_scores = []
for k in k_range:
    knn = KNeighborsClassifier(n_neighbors=k)
    scores = cross_val_score(knn, X, y, cv=10, scoring='accuracy')
    k_scores.append(scores.mean())
```

第6章 线性回归

线性回归模型



代价函数

$$J(\beta_0,\beta_1)=\frac{1}{2m}\sum_{i=1}^{m}\left(\left(\beta_0+\beta_1 x_{obs}^{(i)}\right)-y_{obs}^{(i)}\right)^2$$

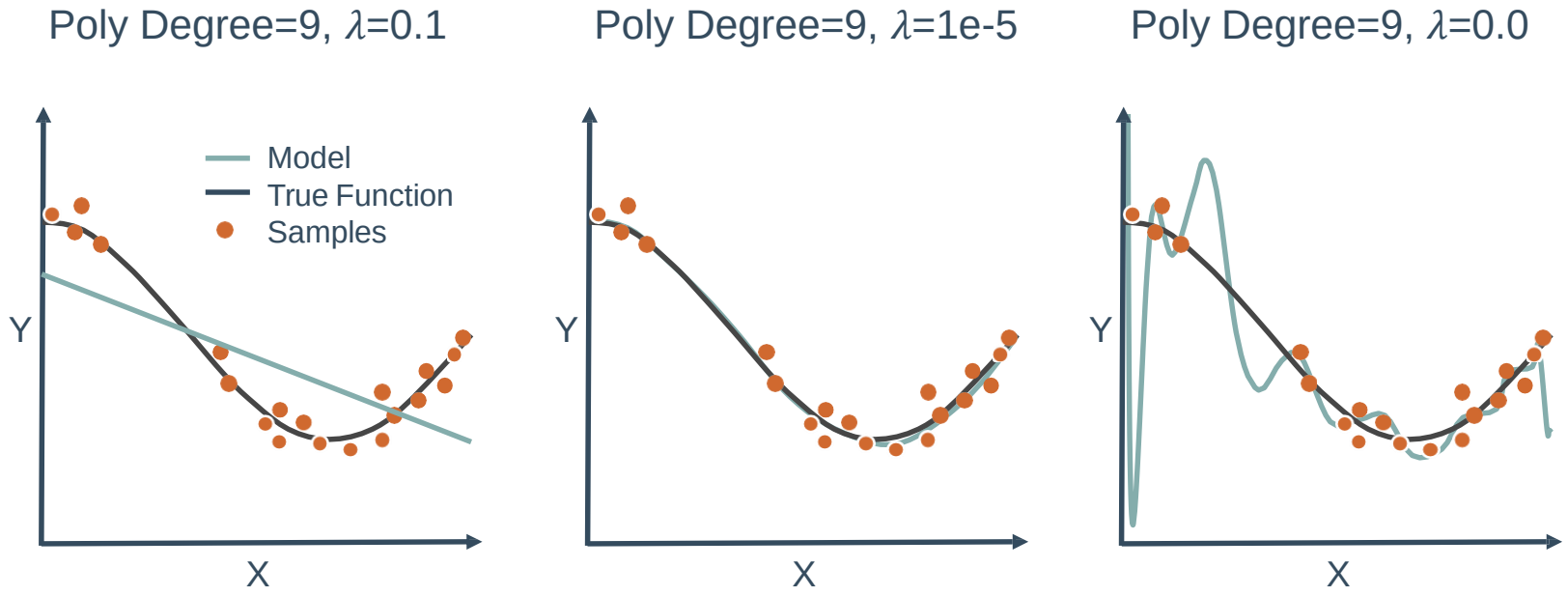
平均绝对误差（Mean Absolute Error, MAE）：

$$\frac{1}{m}\sum_{i=1}^{m}\left|y_\beta\left(x^{(i)}\right)-y_{obs}^{(i)}\right|$$

均方根误差（Root Mean Squared Error, RMSE）：

$$\sqrt{\frac{1}{m}\sum_{i=1}^{m}\left(y_\beta\left(x^{(i)}\right)-y_{obs}^{(i)}\right)^2}$$

正则化（regularization）

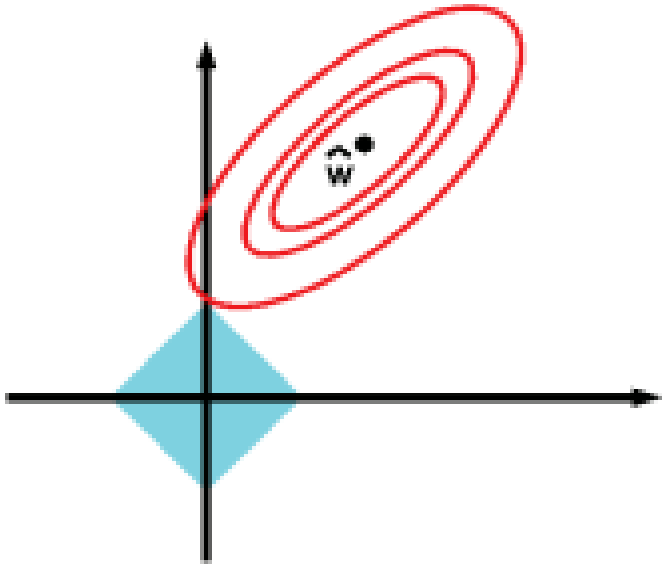


$$J(\beta) = \frac{1}{2m}\sum_{i=1}^{m}\left(y_\beta\left(x^{(i)}\right) - y^{(i)}\right)^2 + \lambda\sum_{j=1}^{n}\beta_j^2$$

套索回归（Lasso Regression）(L1)

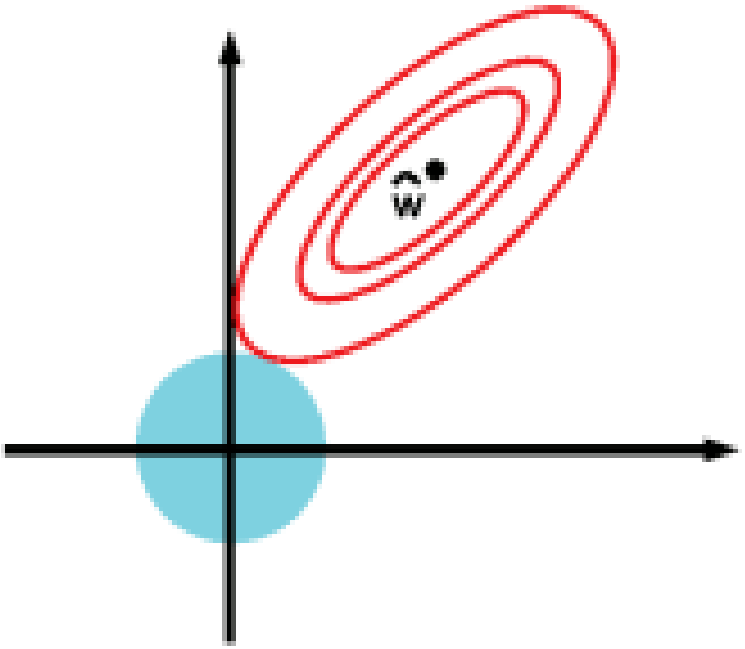
$$J(\beta) = \frac{1}{2m}\sum_{i=1}^{m}\left(y_\beta\left(x^{(i)}\right) - y^{(i)}\right)^2 + \lambda\sum_{j=1}^{n}|\beta_j|$$

- 惩罚项有选择地收缩了某些系数
- 可以被用来做特征选择
- 比岭回归收敛速度慢



岭回归（Ridge Regression）(L2)

$$J(\beta) = \frac{1}{2m}\sum_{i=1}^{m}\left(y_\beta\left(x^{(i)}\right) - y^{(i)}\right)^2 + \lambda\sum_{j=1}^{n}\beta_j^2$$



- 惩罚项收缩了所有系数的大小
- 越大的系数被惩罚得越多，因为惩罚的是平方

ElasticNet正则化

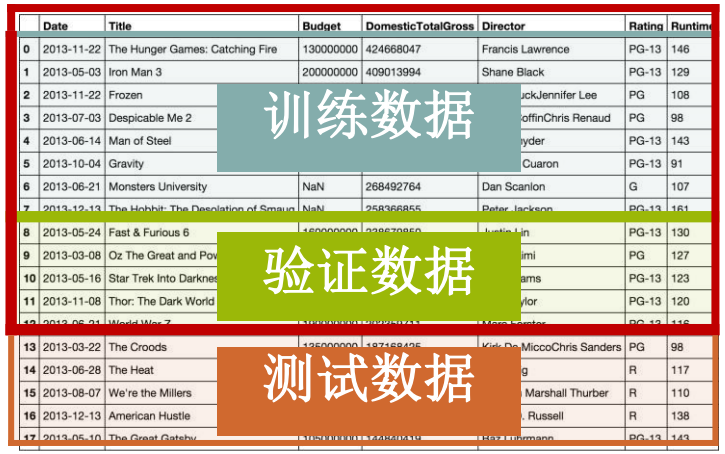
$$J(\beta) = \frac{1}{2m}\sum_{i=1}^{m}\left(y_\beta\left(x^{(i)}\right) - y^{(i)}\right)^2 + \lambda_1 \sum_{j=1}^{n} |\beta_j| + \lambda_2 \sum_{j=1}^{n} \beta_j^2$$

- 岭回归和套索回归的综合，用以平衡稀疏和平滑两个问题
- 需要调节额外的参数，来分配L1和L2正则化惩罚项的比例

超参数及其优化

- 正则化系数（λ_1 和λ_2）是根据经验决定的
- 想让模型泛化---不要使用测试数据集来调节λ_1 和λ_2
- 划分出另一个数据集来调节超参数---验证集（validation set）

用交叉验证来调节λ



	Date	Title	Budget	DomesticTotalGross	Director	Rating	Runtim
0	2013-11-22	The Hunger Games: Catching Fire	130000000	424668047	Francis Lawrence	PG-13	146
1	2013-05-03	Iron Man 3	200000000	409013994	Shane Black	PG-13	129
2	2013-11-22	Frozen			uckJennifer Lee	PG	108
3	2013-07-03	Despicable Me 2			offinChris Renaud	PG	98
4	2013-06-14	Man of Steel			nyder	PG-13	143
5	2013-10-04	Gravity			Cuaron	PG-13	91
6	2013-06-21	Monsters University	NaN	268492764	Dan Scanlon	G	107
7	2013-12-13	The Hobbit: The Desolation of Smaug	NaN	258366855	Peter Jackson	PG-13	161
8	2013-05-24	Fast & Furious 6	160000000	238679850	Justin Lin	PG-13	130
9	2013-03-08	Oz The Great and Pov			imi	PG	127
10	2013-05-16	Star Trek Into Darknes			ams	PG-13	123
11	2013-11-08	Thor: The Dark World			ylor	PG-13	120
12	2013-06-21	World War Z	[illegible]	[illegible]	Marc Forster	PG-13	116
13	2013-03-22	The Croods	135000000	187168425	Kirk DeMiccoChris Sanders	PG	98
14	2013-06-28	The Heat			g	R	117
15	2013-08-07	We're the Millers			Marshall Thurber	R	110
16	2013-12-13	American Hustle			. Russell	R	138
17	2013-05-10	The Great Gatsby	[illegible]	[illegible]	Baz Luhrmann	PG-13	143

岭回归

导入包含回归方法的类：

```
from sklearn.linear_model import RidgeCV
```

创建该类的一个对象：

```
RRcv = RidgeCV(alphas=[1e-3, 1e-2, 1e-1, 1], cv=4)
```

拟合训练数据，并在测试数据上预测：

```
RRcv = RRcv.fit(X_train, y_train)
y_predict = RRcv.predict(X_test)
```

RidgeCV 使用交叉验证自动确定alpha的值

套索回归

导入包含回归方法的类：

```
from sklearn.linear_model import LassoCV
```

创建该类的一个对象：

```
LRcv = LassoCV(alphas=[1e-3, 1e-2, 1e-1, 1], cv=4)
```

拟合训练数据，并在测试数据上预测：

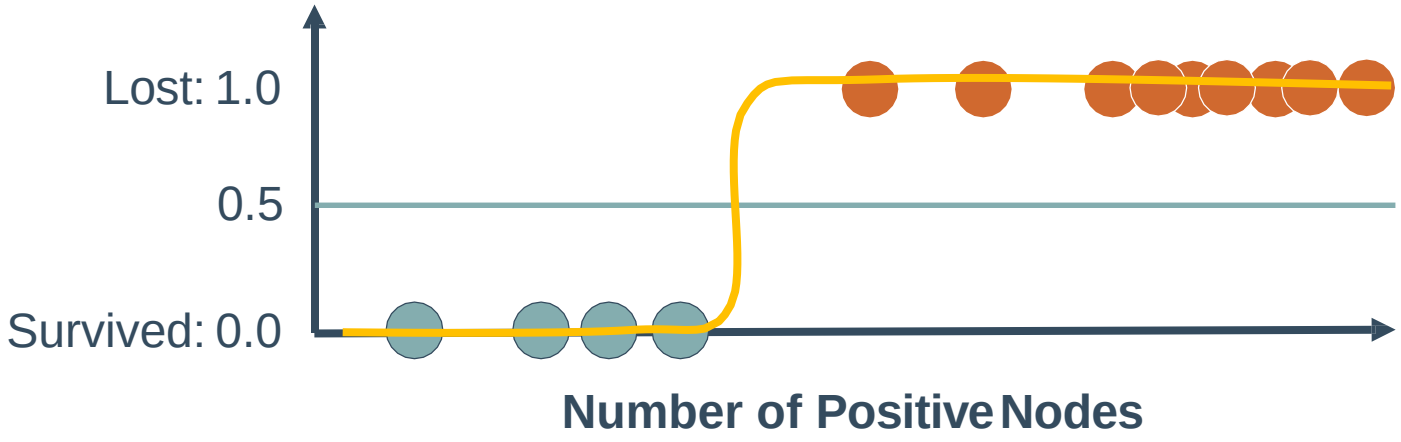
```
LRcv = LRcv.fit(X_train, y_train)
y_predict = LRcv.predict(X_test)
```

LassoCV 使用交叉验证自动确定alpha的值

第7章 逻辑回归

逻辑回归

癌症病人治疗5年之后的状况



$$y_\beta(x) = \frac{1}{1+e^{-(\beta_0 + \beta_1 x + \varepsilon)}}$$

多分类

多分类方法

1. 二分类方法推广到多类
2. 利用二分类学习器解决多分类问题（常用）

- 对问题进行拆分，为拆出的每个二分类任务训练一个分类器
- 对每个分类器的预测结果进行集成以获得最终的多分类结果

拆分策略

- 一对一（One vs. One, OvO）
- 一对其余（One vs. Rest, OvR）
- 多对多（Many vs. Many, MvM）

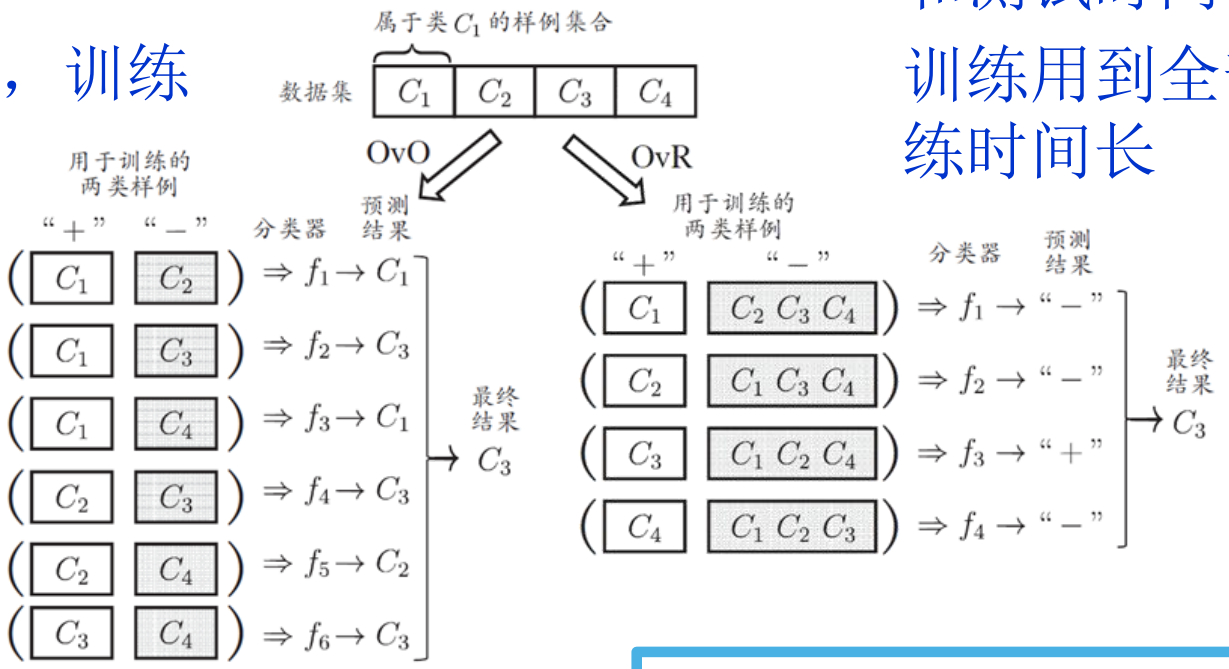
多分类 - 两种策略比较

训练N(N-1)/2个分类器，存储开销和测试时间大

训练只用两个类的样例，训练时间短

训练N个分类器，存储开销和测试时间小

训练用到全部训练样例，训练时间长



预测性能取决于具体数据分布，多数情况下两者差不多

逻辑回归

$$\min_{w} C \sum_{i=1}^{n} \left(-y_i \log(\hat{p}(X_i)) - (1-y_i)\log(1-\hat{p}(X_i))\right) + r(w).$$

导入包含分类方法的类：

```
from sklearn.linear_model import LogisticRegression
```

创建该类的一个实例：

```
LR = LogisticRegression(penalty = '12', C = 10.0)
```

表示正则化强度的倒数

拟合训练数据并预测：

```
LR = LR.fit(X_train, y_train)

y_predict = LR.predict(X_test)
```

使用交叉验证自动确定正则化参数：**LogisticRegressionCV**

LogisticRegression

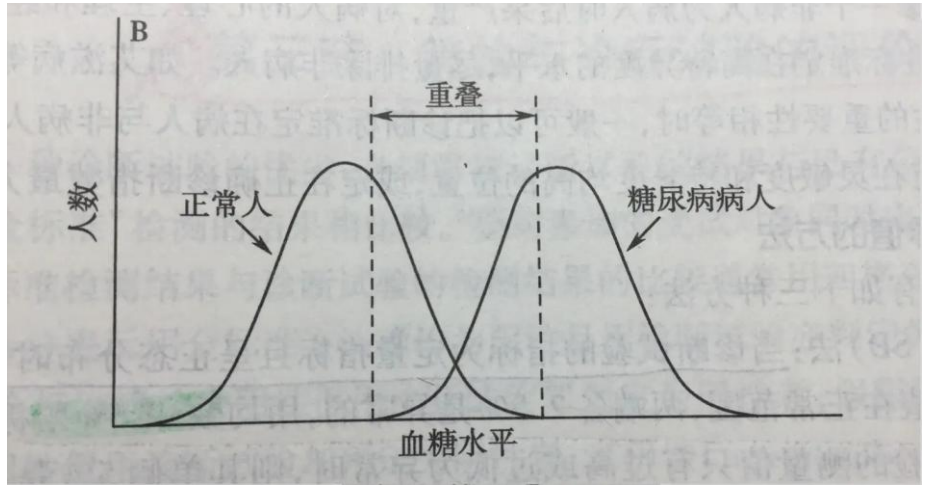
```
LogisticRegression(penalty='l2', dual=False, tol=0.0001, C=1.0,
    fit_intercept=True, intercept_scaling=1, class_weight=None,
    random_state=None, solver='lbfgs', max_iter=100,
    multi_class='auto', verbose=0, warm_start=False, n_jobs=None,
    l1_ratio=None)
```

multi_class : {‘auto’, ‘ovr’, ‘multinomial’}

第8章 分类评价指标

评价指标

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)



$$\text{Accuracy} = \frac{TP + TN}{TP + FN + FP + TN}$$

$$\text{Error} = 1 - \text{Accuracy}$$

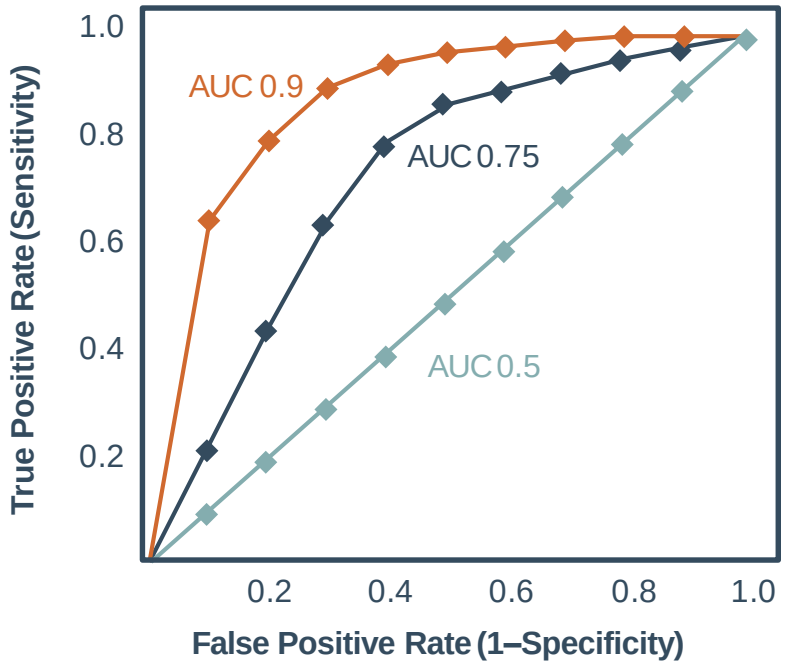
$$\text{Precision} = \frac{TP}{TP + FP}$$

$$\text{Recall or Sensitivity} = \frac{TP}{TP + FN}$$ 漏诊率

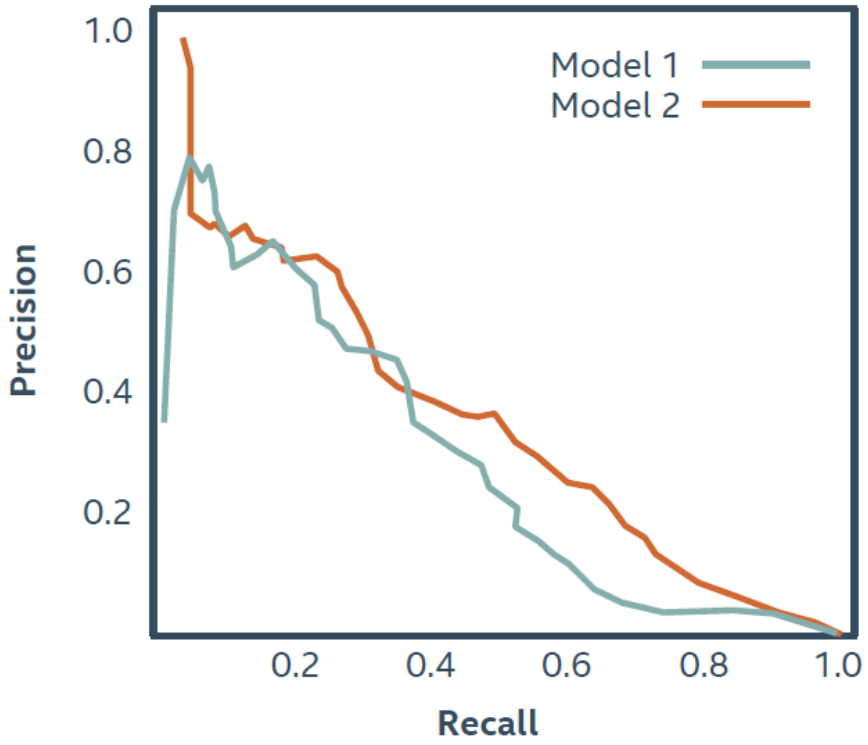
$$F1 = 2 \times \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}}$$

$$\text{Specificity} = \frac{TN}{FP + TN}$$ 误诊率

曲线



衡量ROC曲线下的面积



P-R衡量查准率与查全率之间的权衡

第9章 朴素贝叶斯

朴素贝叶斯分类器

$$P(c \mid \mathbf{x}) = \frac{P(c)P(\mathbf{x} \mid c)}{P(\mathbf{x})} = \frac{P(c)}{P(\mathbf{x})} \prod_{i=1}^{d} P(x_i \mid c)$$

由于对所有类别来说$P(x)$相同，因此有贝叶斯判定准则

$$h_{nb}(\mathbf{x}) = \underset{c \in y}{\operatorname{argmax}}\, P(c) \prod_{i=1}^{d} P(x_i \mid c)$$

案例：预测打网球

$$\underset{k \in \{1, \dots K\}}{argmax} P(C_k) \prod_{i=1}^{n} P(X_i | C_k)$$

预测下面情境下是否打网球：

x'=(Outlook= Overcast Temperature=Cool, Humidity=High, Wind=Strong)

Day	Outlook	Temperature	Humidity	Wind	PlayTennis
D1	Sunny	Hot	High	Weak	No
D2	Sunny	Hot	High	Strong	No
D3	Overcast	Hot	High	Weak	Yes
D4	Rain	Mild	High	Weak	Yes
D5	Rain	Cool	Normal	Weak	Yes
D6	Rain	Cool	Normal	Strong	No
D7	Overcast	Cool	Normal	Strong	Yes
D8	Sunny	Mild	High	Weak	No
D9	Sunny	Cool	Normal	Weak	Yes
D10	Rain	Mild	Normal	Weak	Yes
D11	Sunny	Mild	Normal	Strong	Yes
D12	Overcast	Mild	High	Strong	Yes
D13	Overcast	Hot	Normal	Weak	Yes
D14	Rain	Mild	High	Strong	No

Feature	Play=Yes	Play=No
Outlook=Sunny	2/9	3/5
Temperature=Cool	3/9	1/5
Humidity=High	3/9	4/5
Wind=Strong	3/9	3/5
Overall Label	**9/14**	**5/14**
Probability	**0.0053**	**0.0206**

拉普拉斯平滑

问题：有些类别里没有的特征将会导致这些条件概率值为"0"

$$P(C|X) = \overset{0}{\boxed{P(X_1|C)}} * P(X_2|C) * P(C)$$

解决方案：

$$\hat{P}(c) = \frac{|D_c| + 1}{|D| + N}$$

$$\hat{P}(x_i \mid c) = \frac{|D_{c,x_i}| + 1}{|D| + N_i}$$

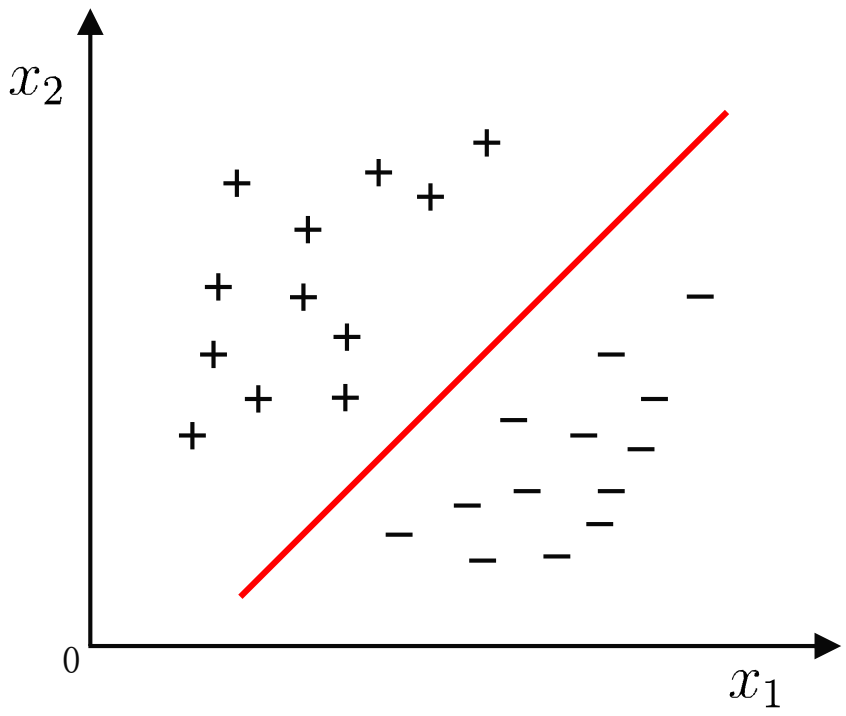
朴素贝叶斯模型的类型

朴素贝叶斯模型	数据类型
贝努利模型	二值 (T/F)
多项式模型	离散值 (如，计数)
高斯模型	连续值

不同朴素贝叶斯模型的区别，主要在于它们对概率分布 $P(X_i/C)$所做的不同假设。

第10章 支持向量机

线性模型：在样本空间中寻找一个**超平面**，将不同类别的样本分开.



$$\begin{cases} \omega^T x + b > 0, y_i = 1 \\ \omega^T x + b < 0, y_i = -1 \end{cases}$$

- 间隔
- 支持向量
- 核函数
- 软间隔

使用核函数的SVM的语法

$$\min_{\boldsymbol{w},b} \ \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{i=1}^{m}\max\left(0, 1 - y_i(\boldsymbol{w}^\top\phi(\boldsymbol{x}_i) + b)\right)$$

导入包含分类方法的类：

```
from sklearn.svm import SVC
```

创建该类的一个对象：

```
rbfSVC = SVC(kernel='rbf', gamma=1.0, C=10.0)
```

"C" 是错误项的惩罚力度

设置核函数，及其相应参数(gamma).

$$\gamma = \frac{1}{2\sigma^2}$$

拟合训练数据，并预测：

```
rbfSVC = rbfSVC.fit(X_train, y_train)
y_predict = rbfSVC.predict(X_test)
```

- kernel:核函数类型，str类型，默认为‘rbf’。可选参数为：
  - ‘linear’：线性核函数
  - ‘poly’：多项式核函数
  - ‘rbf’：径向基核函数/高斯核
  - ‘sigmod’：sigmod核函数
  - ‘precomputed’：核矩阵。precomputed表示自己提前计算好核函数矩阵，这时候算法内部就不再用核函数去计算核矩阵，而是直接用该n*n的核矩阵。

- gamma:核函数系数，float类型，可选参数，默认为auto。只对’rbf’,’poly’ ,’sigmod’有效。如果gamma为auto，代表其值为样本特征数的倒数，即1/n_features。（即核函数的带宽，超圆的半径）。gamma越大，σ越小，使得高斯分布又高又瘦，造成模型只能作用于支持向量附近，可能导致过拟合；反之，gamma越小，σ越大，高斯分布会过于平滑，在训练集上分类效果不佳，可能导致欠拟合。

class **sklearn.svm.SVC**(*C=1.0, kernel='rbf', degree=3, gamma='auto', coef0=0.0, shrinking=True, probability=False, tol=0.001, cache_size=200, class_weight=None, verbose=False, max_iter=-1, decision_function_shape='ovr', random_state=None*)

$$\min_{\boldsymbol{w},b} \ \frac{1}{2}\|\boldsymbol{w}\|^2 + C\sum_{i=1}^{m}\max\left(0, 1 - y_i(\boldsymbol{w}^\top\phi(\boldsymbol{x}_i) + b)\right)$$

- decision_function_shape ：决策函数类型，可选参数’ovo’和’ovr’，默认为’ovr’。原始的SVM只适用于二分类问题，如果要将其扩展到多类分类，就要采取一定的融合策略， ‘ovo’ 一对一，为one v one，即将类别两两之间进行划分，用二分类的方法模拟多分类的结果，决策所使用的返回的是（样本数，类别数*(类别数-1)/2）； ‘ovr’ 一对多，为one v rest，即一个类别与其他类别进行划分，返回的是(样本数，类别数）。

GridSearchCV

*class sklearn.model_selection.GridSearchCV(estimator, param_grid, scoring=None, fit_params=None, n_jobs=1, iid=True, refit=True, cv=None, verbose=0, pre_dispatch='2*n_jobs', error_score='raise', return_train_score='warn')*

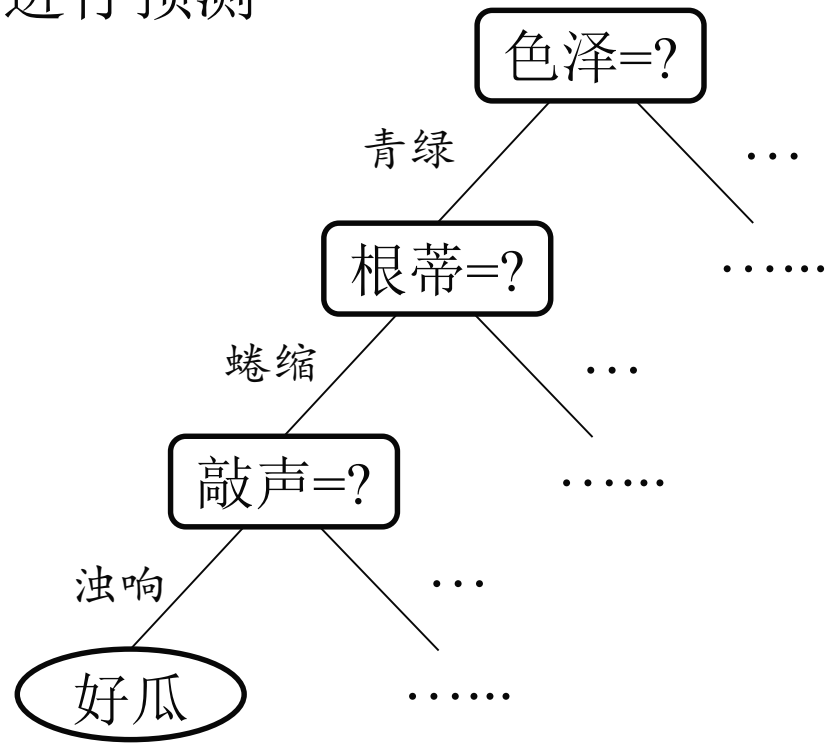
用于选取模型的最优超参数

网格搜索针对超参数组合列表中的每一个组合，实例化给定的模型，做cv次交叉验证，将平均得分最高的超参数组合作为最佳的选择，返回模型对象。

第11章 决策树

基本流程

决策树基于树结构来进行预测



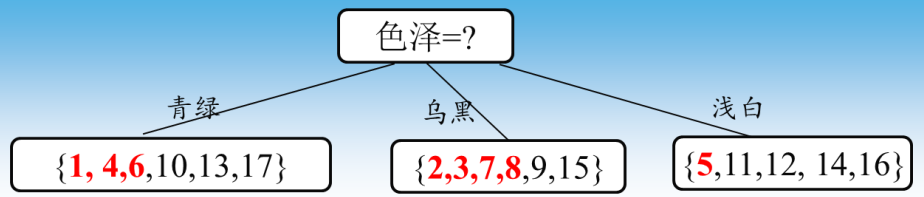
- 信息增益
- 基尼指数

信息增益实例

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否

该数据集包含17个训练样本，$|\mathcal{Y}|=2$，其中正例占 $p_1=\frac{8}{17}$，反例占 $p_2=\frac{9}{17}$，计算得到根结点的信息熵为

$$\mathrm{Ent}(D)=-\sum_{k=1}^{2}p_k\log_2 p_k=-(\frac{8}{17}\log_2\frac{8}{17}+\frac{9}{17}\log_2\frac{9}{17})=0.998$$

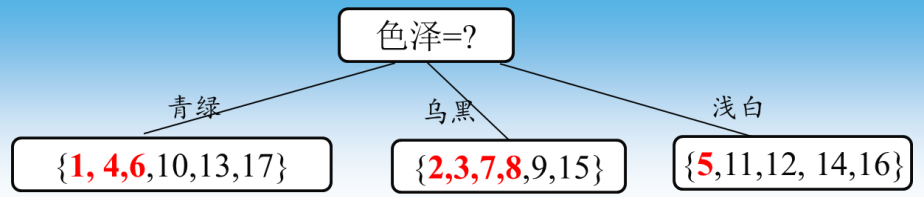


以属性“色泽”为例，其对应的3 个数据子集分别为 D^1（色泽=青绿），D^2（色泽=乌黑），D^3（色泽=浅白）

子集 D^1 包含编号为 $\{1,4,6,10,13,17\}$ 的6 个样例，其中正例占 $p_1=\frac{3}{6}$ ，反例占 $p_2=\frac{3}{6}$ 信息熵为：

$$\mathrm{Ent}(D^1)=-\left(\frac{3}{6}\log_2\frac{3}{6}+\frac{3}{6}\log_2\frac{3}{6}\right)=1.000$$

编号	色泽	根蒂	敲声	纹理	脐部	触感	好瓜
1	青绿	蜷缩	浊响	清晰	凹陷	硬滑	是
2	乌黑	蜷缩	沉闷	清晰	凹陷	硬滑	是
3	乌黑	蜷缩	浊响	清晰	凹陷	硬滑	是
4	青绿	蜷缩	沉闷	清晰	凹陷	硬滑	是
5	浅白	蜷缩	浊响	清晰	凹陷	硬滑	是
6	青绿	稍蜷	浊响	清晰	稍凹	软粘	是
7	乌黑	稍蜷	浊响	稍糊	稍凹	软粘	是
8	乌黑	稍蜷	浊响	清晰	稍凹	硬滑	是
9	乌黑	稍蜷	沉闷	稍糊	稍凹	硬滑	否
10	青绿	硬挺	清脆	清晰	平坦	软粘	否
11	浅白	硬挺	清脆	模糊	平坦	硬滑	否
12	浅白	蜷缩	浊响	模糊	平坦	软粘	否
13	青绿	稍蜷	浊响	稍糊	凹陷	硬滑	否
14	浅白	稍蜷	沉闷	稍糊	凹陷	硬滑	否
15	乌黑	稍蜷	浊响	清晰	稍凹	软粘	否
16	浅白	蜷缩	浊响	模糊	平坦	硬滑	否
17	青绿	蜷缩	沉闷	稍糊	稍凹	硬滑	否



同理，D^2 ，D^3的信息熵为：

$$\mathrm{Ent}(D^2) = -(\tfrac{4}{6}\log_2\tfrac{4}{6} + \tfrac{2}{6}\log_2\tfrac{2}{6}) = 0.918$$

$$\mathrm{Ent}(D^3) = -(\tfrac{1}{5}\log_2\tfrac{1}{5} + \tfrac{4}{5}\log_2\tfrac{4}{5}) = 0.722$$

属性“色泽”的信息增益为

$$\begin{aligned}\mathrm{Gain}(D,\text{色泽}) &= \mathrm{Ent}(D) - \sum_{v=1}^{3}\frac{|D^v|}{|D|}\mathrm{Ent}(D^v)\\ &= 0.998 - (\tfrac{6}{17}\times 1.000 + \tfrac{6}{17}\times 0.918 + \tfrac{5}{17}\times 0.722) \ = 0.109\end{aligned}$$

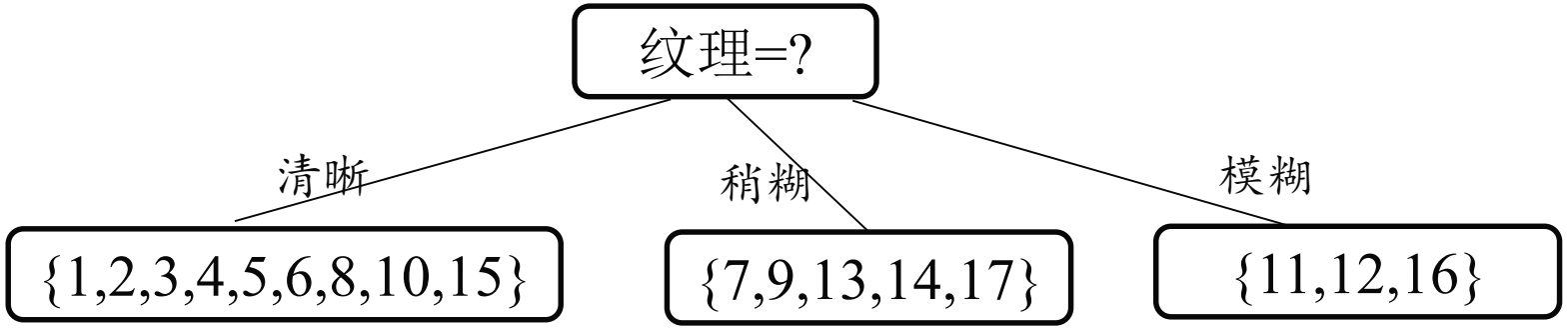
类似的，其它属性的信息增益为

$$\mathrm{Gain}(D, \text{根蒂}) = 0.143 \qquad \mathrm{Gain}(D, \text{敲声}) = 0.141$$

$$\mathrm{Gain}(D, \text{纹理}) = 0.381 \qquad \mathrm{Gain}(D, \text{脐部}) = 0.289$$

$$\mathrm{Gain}(D, \text{触感}) = 0.006$$

显然，属性“纹理”的信息增益最大，其被选为划分属性



划分选择-基尼指数

数据集 D 的纯度可用“基尼值”来度量

$$\mathrm{Gini}(D) = \sum_{k=1}^{|\mathcal{Y}|} \sum_{k' \neq k} p_k p_{k'} = 1 - \sum_{k=1}^{|\mathcal{Y}|} p_k^2$$

反映了从 D 中随机抽取两个样本，其类别标记不一致的概率

$\mathrm{Gini}(D)$ 越小，数据集 D 的纯度越高

属性 a 的基尼指数定义为：

$$\mathrm{Gini_index}(D, a) = \sum_{v=1}^{V} \frac{|D^v|}{|D|} \mathrm{Gini}(D^v)$$

应选择那个使划分后基尼指数最小的属性作为最优划分属性，即

$$a_* = \underset{a \in A}{\operatorname{argmin}}\ \mathrm{Gini_index}(D, a)$$

CART [Breiman et al., 1984]采用“基尼指数”来选择划分属性

剪枝处理

为什么剪枝

- “剪枝”是决策树学习算法对付“过拟合”的主要手段
- 可通过“剪枝”来一定程度避免因决策分支过多，以致于把训练集自身的一些特点当做所有数据都具有的一般性质而导致的过拟合

剪枝的基本策略

- 预剪枝
- 后剪枝

判断决策树泛化性能是否提升的方法

留出法：预留一部分数据用作“验证集”以进行性能评估

决策树分类器

导入包含分类方法的类：

```
from sklearn.tree import DecisionTreeClassifier
```

创建该类的一个对象：

```
DTC = DecisionTreeClassifier(criterion='gini’,
                max_features=10, max_depth=5)
```

拟合训练数据，并预测：

```
DTC = DTC.fit(X_train, y_train)
y_predict = DTC.predict(X_test)
```

用交叉验证来调参，使用DecisionTreeRegressor做回归

第12章 集成学习

Bagging

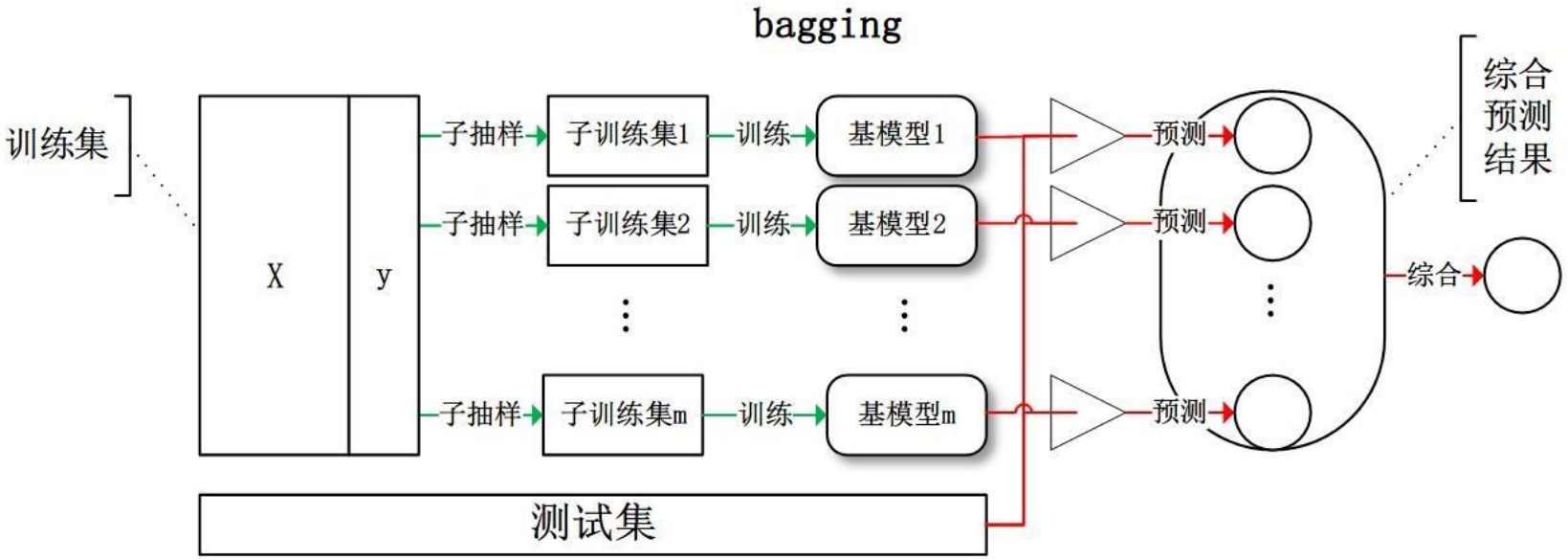
个体学习器不存在强依赖关系

并行化生成

自助采样法

随机森林(Random Forest，简称RF)是bagging的一个扩展变种

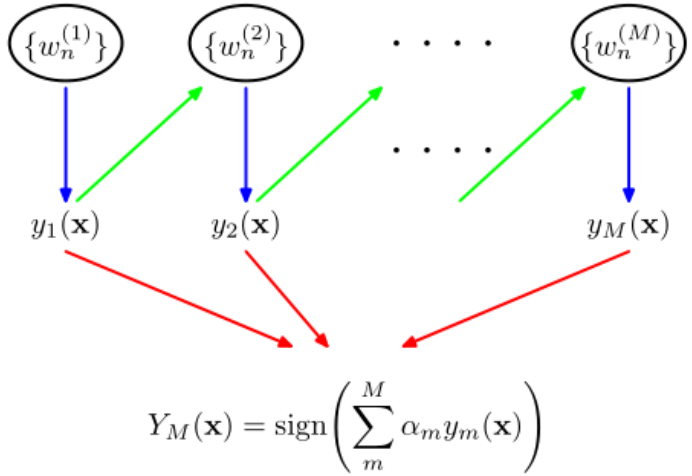
- 采样的随机性
- 属性选择的随机性



Boosting

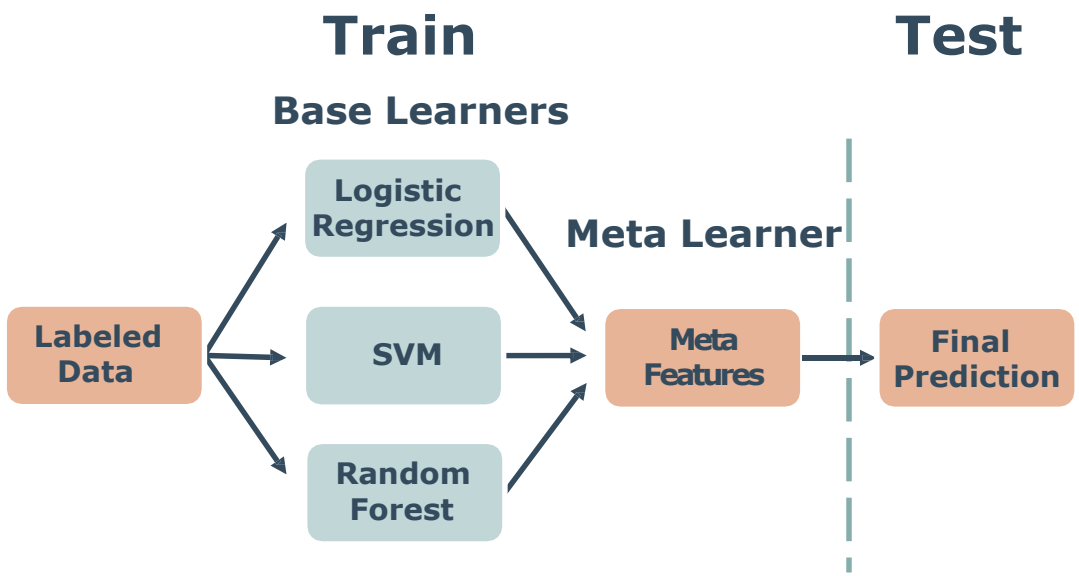
1. 先从初始训练集训练出一个基学习器。
2. 再根据基学习器的表现对训练样本分布进行调整，使得先前基学习器做错的训练样本在后续受到更多关注。
3. 然后基于调整后的样本分布来训练下一个基学习器。
4. 重复2，3步，直到基学习器的数目达到事先指定的值T。
5. 最后将T个基学习器进行加权结合。

AdaBoost
梯度提升树



Stacking：结合多个异构的分类器

- 不同类型的模型可以结合起来构建stacked模型
- 类似bagging，但不局限于决策树



第13章 聚类

k均值算法

给定数据集 $D=\{x_1,x_2,\cdots,x_m\}$ ， k均值算法针对聚类所得簇划分 $C=\{C_1,C_2,\cdots,C_k\}$ 最小化平方误差

$$E=\sum_{i=1}^{k}\sum_{x\in C_j}\|x-\mu_i\|_2^2$$

其中，μ_i 是簇 C_i 的均值向量。

$$u_i=\frac{1}{|C_i|}\sum_{x\in C_i}x$$

E 值在一定程度上刻画了簇内样本围绕簇均值向量的紧密程度，E 值越小，则簇内样本相似度越高。

- 算法流程（迭代优化）：

初始化每个簇的均值向量

repeat

1. （更新）簇划分；
2. 计算每个簇的均值向量

until 当前均值向量均未更新

k均值算法实例

以表中数据集为例，演示k均值算法的学习过程。将编号为 i 的样本称为 x_i .

编号	密度	含糖率	编号	密度	含糖率	编号	密度	含糖率
1	0.697	0.460	11	0.245	0.057	21	0.748	0.232
2	0.774	0.376	12	0.343	0.099	22	0.714	0.346
3	0.634	0.264	13	0.639	0.161	23	0.483	0.312
4	0.608	0.318	14	0.657	0.198	24	0.478	0.437
5	0.556	0.215	15	0.360	0.370	25	0.525	0.369
6	0.403	0.237	16	0.593	0.042	26	0.751	0.489
7	0.481	0.149	17	0.719	0.103	27	0.532	0.472
8	0.437	0.211	18	0.359	0.188	28	0.473	0.376
9	0.666	0.091	19	0.339	0.241	29	0.725	0.445
10	0.243	0.267	20	0.282	0.257	30	0.446	0.459

k均值算法实例

假定聚类簇数k =3，算法开始时，随机选择3个样本 x_6, x_{12}, x_{27} 作为初始均值向量，$\mu_1 = (0.403; 0.237), \mu_2 = (0.343; 0.099), \mu_3 = (0.533; 0.472)$

考察样本 $x_1 = (0.697; 0.460)$，它与当前均值向量 μ_1, μ_2, μ_3的距离分别为0.369，0.506，0.166，因此 x_1 将被划入簇 C_3 中。类似的，对数据集中的所有样本考察一遍后，可得当前簇划分为

$$C_1 = \{x_5, x_6, x_7, x_8, x_9, x_{10}, x_{13}, x_{14}, x_{15}, x_{17}, x_{18}, x_{19}, x_{20}, x_{23}\}$$
$$C_2 = \{x_{11}, x_{12}, x_{16}\}$$
$$C_3 = \{x_1, x_2, x_3, x_4, x_{21}, x_{22}, x_{24}, x_{25}, x_{26}, x_{27}, x_{28}, x_{29}, x_{30}\}$$

于是，可以分别求得新的均值向量

$$\mu_1' = (0.473; 0.214), \mu_2' = (0.394; 0.066), \mu_3' = (0.623; 0.388)$$

不断重复上述过程。

K-Means的语法

导入包含聚类方法的类：

```
from sklearn.cluster import KMeans
```

创建该类的一个对象：

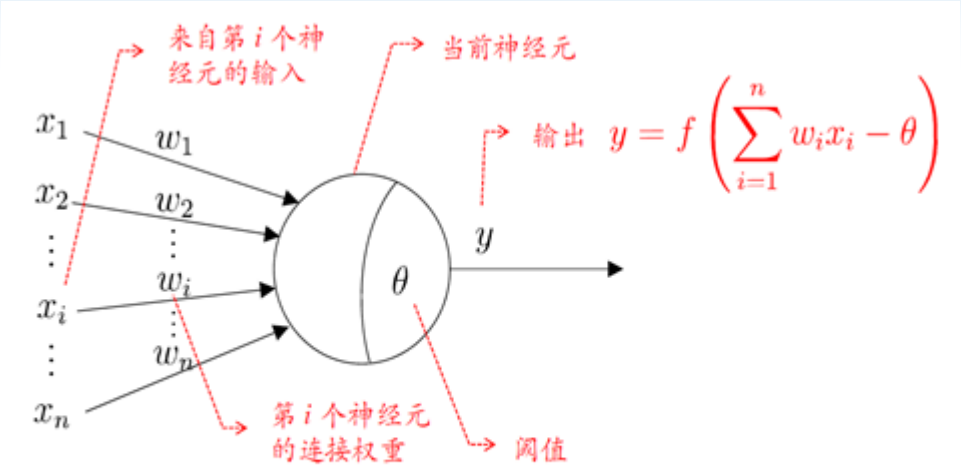
```
kmeans = KMeans(n_clusters=3,
                init='k-means++')
```

拟合数据，并在新数据上预测聚簇：

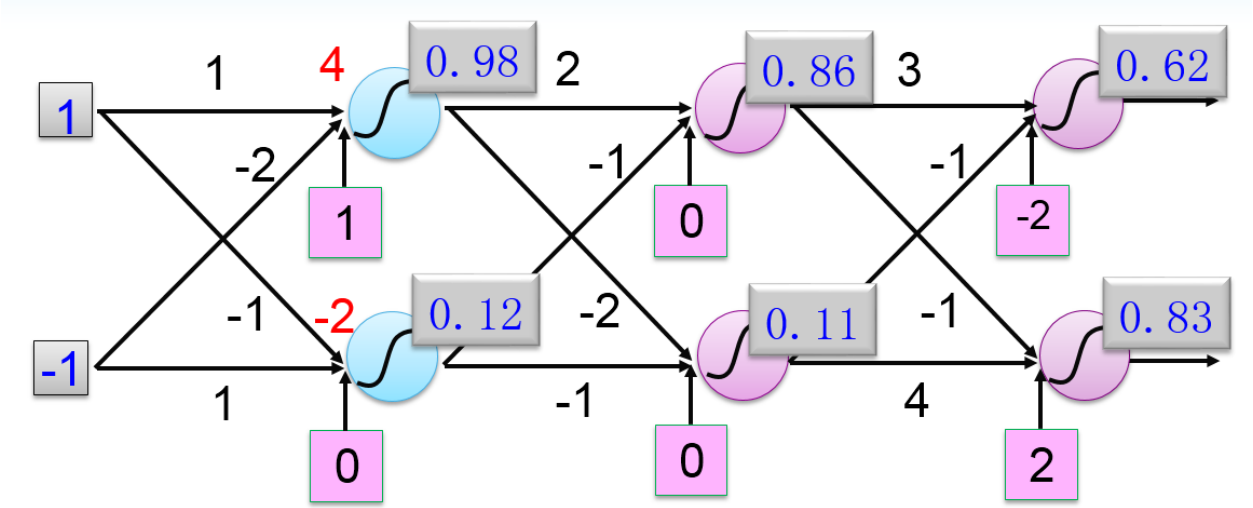
```
kmeans = kmeans.fit(X1)
y_predict = kmeans.predict(X2)
```

第14章 深度学习

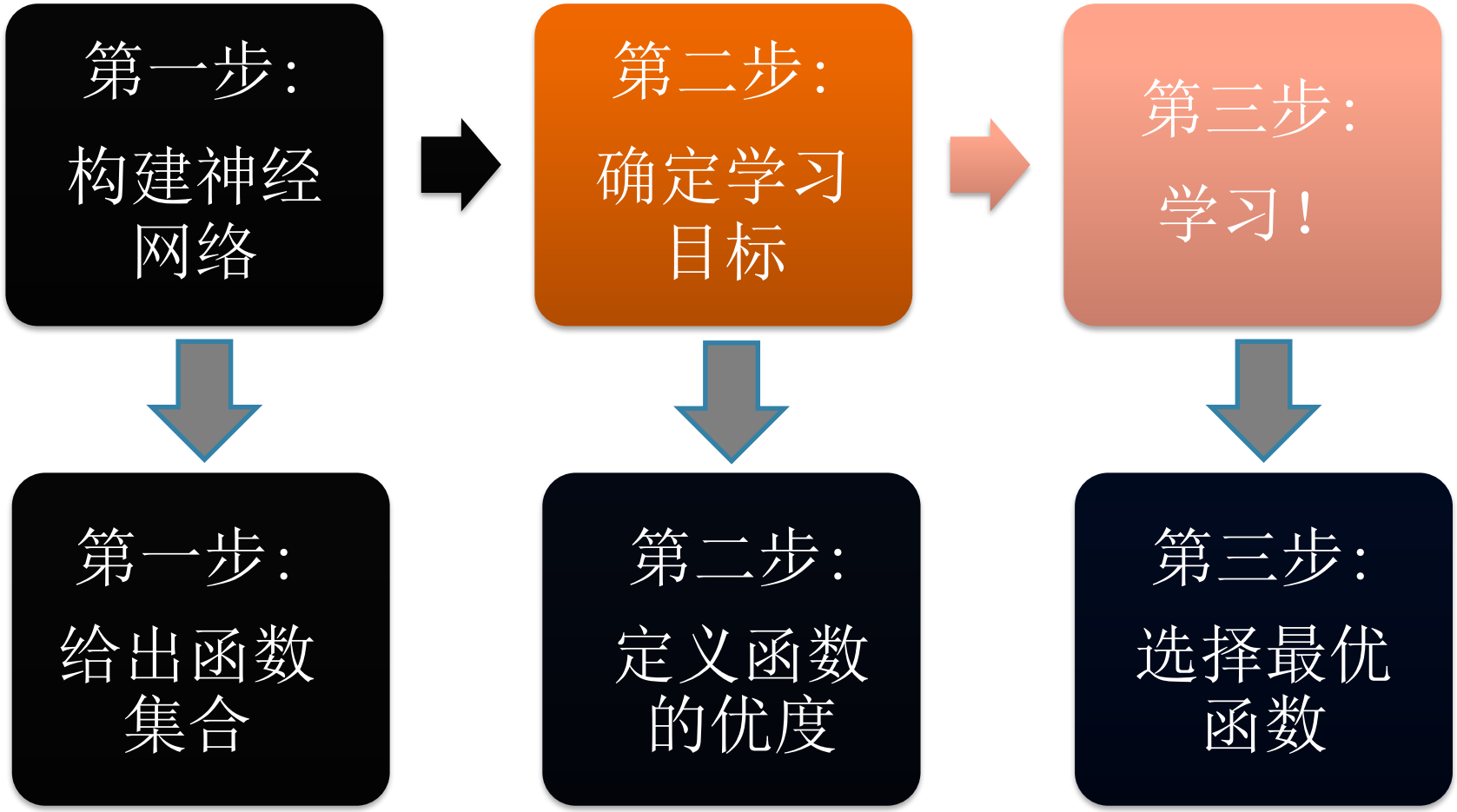
- M-P神经元模型
- 多层感知机网络（前馈神经网络）



M-P 神经元模型

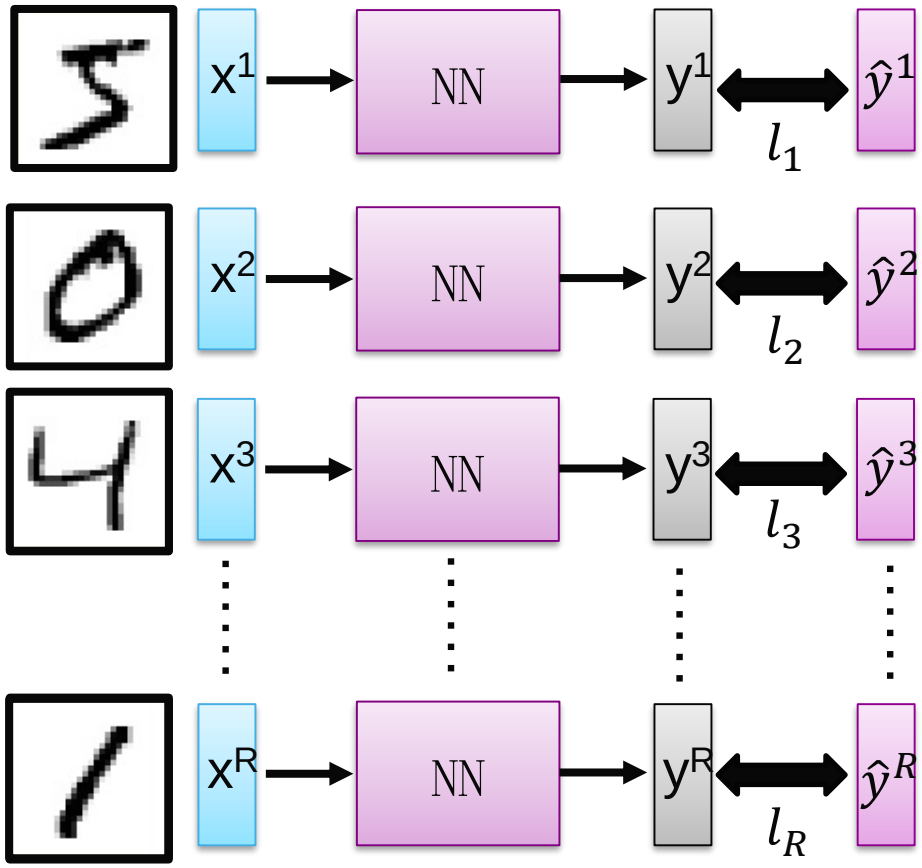


深度学习三部曲


第一步：
构建神经网络
第二步：
确定学习目标
第三步：
学习！
第一步：
给出函数集合
第二步：
定义函数的优度
第三步：
选择最优函数

损失函数

对于所有的训练数据…



整体损失:

$$L = \sum_{r=1}^{R} l_r$$

尽可能小

在函数集中找到使整体损失L最小的一个函数

找到能最小化整体损失L的一组网络参数 $\boldsymbol{\theta}^*$

梯度下降法

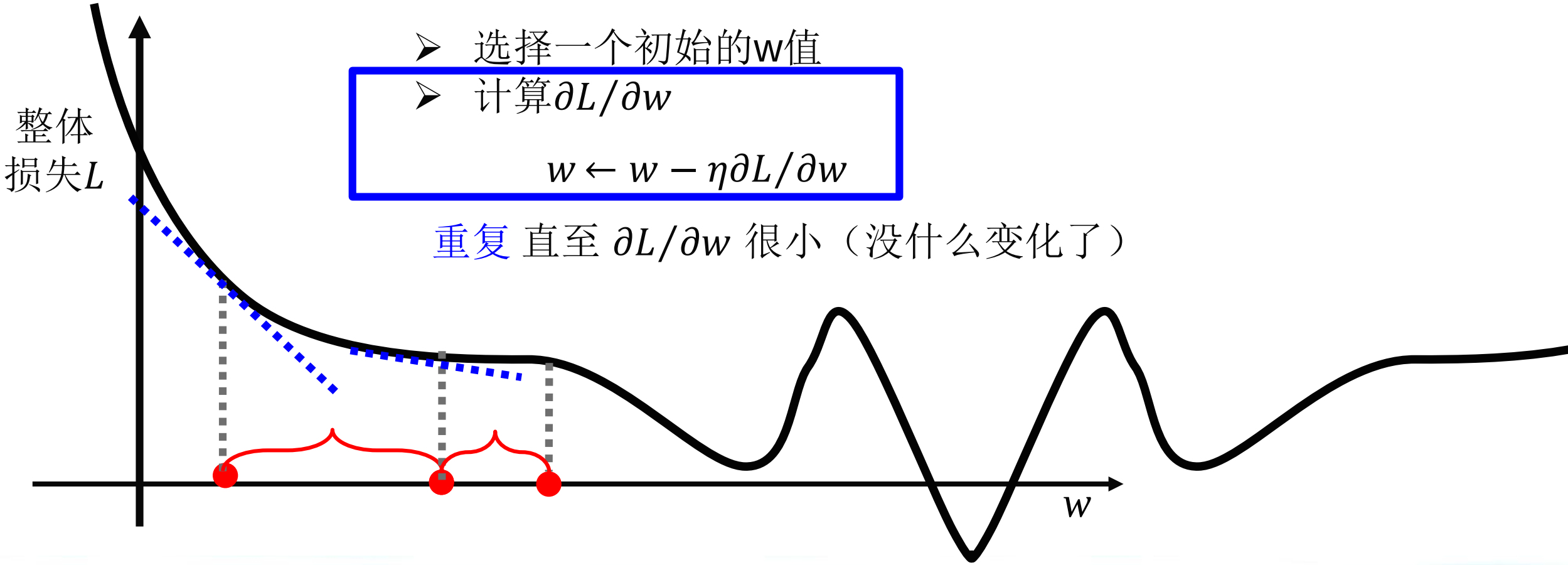
网络参数 θ
$= \{w_1, w_2, \cdots, b_1, b_2, \cdots\}$

找到能最小化整体损失L的一组网络参数 $\boldsymbol{\theta}^*$

- 选择一个初始的w值
- 计算$\partial L/\partial w$

$$w \leftarrow w - \eta \partial L/\partial w$$

重复 直至 $\partial L/\partial w$ 很小（没什么变化了）



BP 学习算法

BP算法基于**梯度下降策略**，以目标的负梯度方向对参数进行调整．对误差 E_k，给定学习率 η

$$\triangle w_{hj} = -\eta \frac{\partial E_k}{\partial w_{hj}}$$

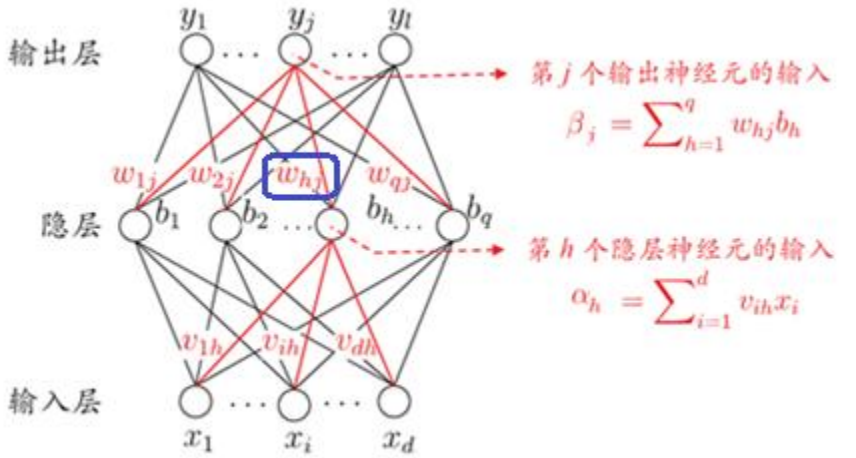
$$E_k = \frac{1}{2}\sum_{j=1}^{l}(\hat{y}_j^k - y_j^k)^2$$

$$\hat{y}_j^k = f(\beta_j - \theta_j),\ \beta_j = \sum_{h=1}^{q} w_{hj} b_h$$

$$\frac{\partial E_k}{\partial w_{hj}} = \boxed{\frac{\partial E_k}{\partial \hat{y}_j^k}} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \cdot \boxed{\frac{\partial \beta_j}{\partial w_{hj}}}$$

$$(\hat{y}_j^k - y_j^k) \qquad f'(\beta_j - \theta_j) \qquad b_h$$

$$\hat{y}_j^k(1 - \hat{y}_j^k)$$

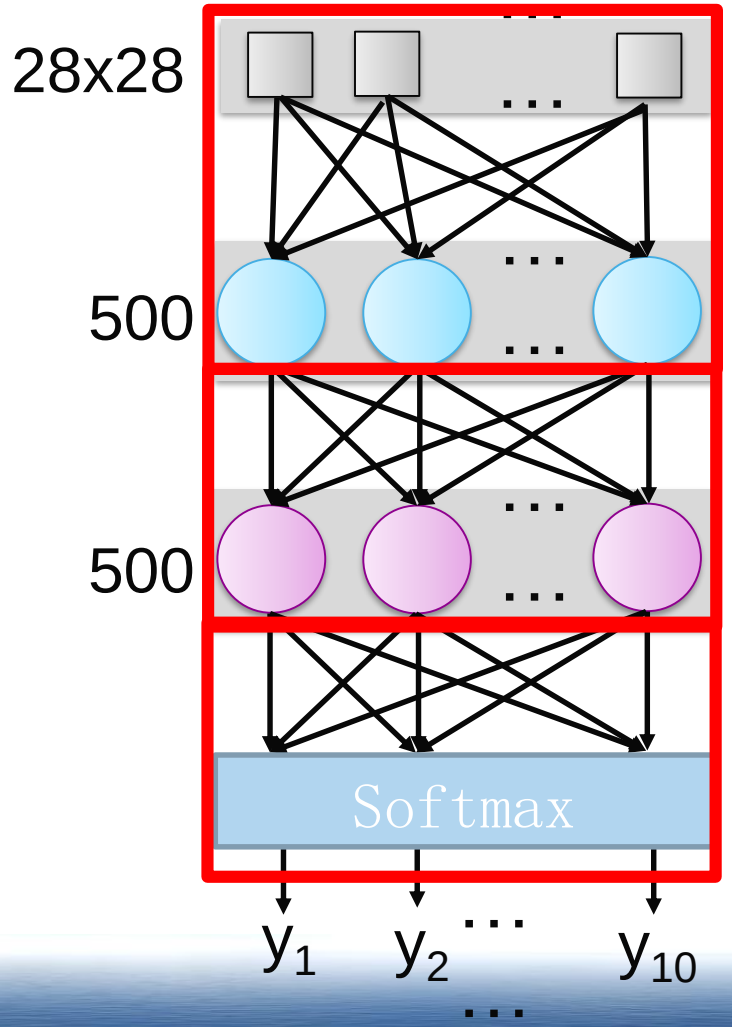


$$\begin{aligned} g_j &= -\frac{\partial E_k}{\partial \hat{y}_j^k} \cdot \frac{\partial \hat{y}_j^k}{\partial \beta_j} \\ &= -(\hat{y}_j^k - y_j^k) f'(\beta_j - \theta_j) \\ &= \hat{y}_j^k(1 - \hat{y}_j^k)(y_j^k - \hat{y}_j^k) \end{aligned}$$

$$\triangle w_{hj} = \eta g_j b_h$$

Keras实例

第一步：构建神经网络 → 第二步：确定学习目标 → 第三步：学习！




```
from keras.models import Sequential
from keras.layers import Dense,Activation

model = Sequential()
```

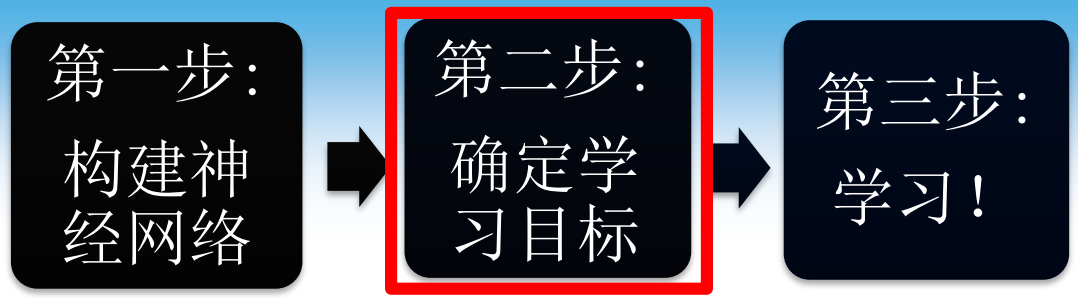
```
model.add(Dense(500, input_dim=784))
model.add(Activation('sigmoid'))
```

```
model.add(Dense(500))
model.add(Activation('sigmoid'))
```

```
model.add(Dense(10))
model.add(Activation('softmax'))
```

```
model.summary()
```

Keras




```
model.compile(loss = 'mean_squared_error',
              optimizer = ‘sgd',
              metrics = ['accuracy'])
```

- loss：损失函数

  mean_squared_error、categorical_crossentropy…..
- optimizer：优化器

  sgd、rmsprop、adam……
- metrics：评价指标

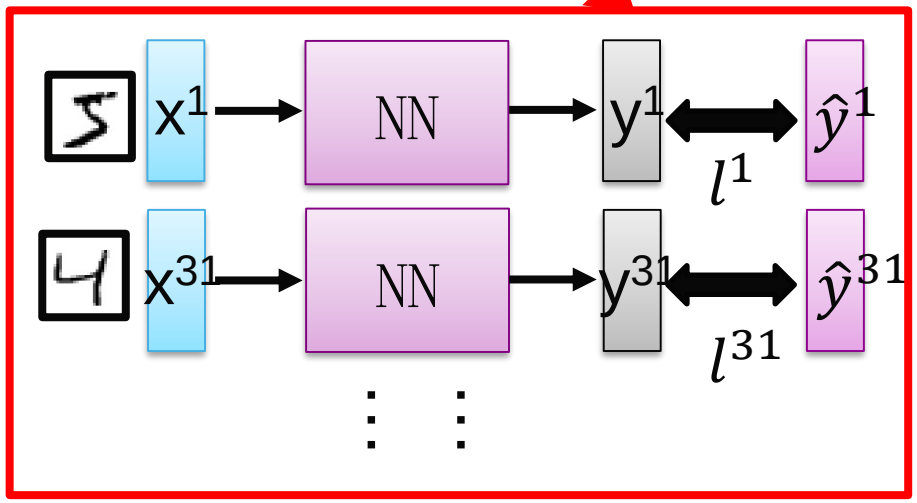
Keras

第一步：构建神经网络 → 第二步：确定学习目标 → 第三步：学习！

```
model.fit(X_train,  Y_train,                    batch_size=100, epochs=20)
```

训练数据（图像）　类别标签（数字）

Mini-batch



每批里有100个样例数据

- 用第一批数据
  $L' = l^1 + l^{31} + \cdots$
  修改一次参数
- 用第二批数据
  $L'' = l^2 + l^{16} + \cdots$
  修改一次参数
  ⋮
- 直到所有批的数据都使用过

重复20次　一个epoch

深度学习技巧

YES

NO

Good Results on Testing Data?

- Dropout
- 正则化（Regularization）
- 提前终止（Early Stopping）

YES

NO

Good Results on Training Data?

- 新的激活函数
- 合适的损失函数
- 自适应的学习率、动量

卷积神经网络

- 卷积
- 池化
- 激活函数relu
- softmax